ESIC University

Proyecto de Implantacion de Big Data I

Documento de Ejecucion de Pruebas

Tema 7 - Documentacion del Proyecto

Curso 2025 - 2026

1. Informacion del Documento

Proyecto	API de Predicción de Productividad Estudiantil
Equipo	Laura Galán, Daniela Muñiz, Patricia Rivera, Mariela Rivero
Fecha de ejecucion	10/05/2026
Version del documento	V1.0

2. Pruebas Funcionales

Las pruebas funcionales verifican que el sistema se comporta segun los requisitos definidos. Cada prueba debe tener un resultado esperado claro y un resultado real obtenido.

ID	Tipo	Descripcion de la prueba	Resultado esperado	Resultado obtenido	Estado
Requisitos Funcionales					
PF01	RF1	Carga del dataset - Al ejecutar Proyecto_Principal.py el sistema carga el CSV con 20.000 registros correctamente	Dataset cargado sin errores. Mensaje de confirmación en consola	CSV cargado correctamente. DataFrame con 20.000 filas y columnas esperadas	OK
PF02	RF2	GET /estudiantes - Consulta listado paginado de estudiantes	JSON con array de estudiantes e información de paginación	Respuesta 200 con registros y objeto paginación correcto	OK
PF03	RF2	GET /estudiantes?nivel=ALTA - Filtrado por nivel de productividad	Solo estudiantes con nivel_productividad=ALTA	Respuesta 200 con registros filtrados correctamente	OK
PF04	RF3	GET /estudiante/1 - Obtener estudiante por ID existente	JSON con todos los datos del estudiante con id=1	Respuesta 200 con datos completos del estudiante	OK
PF05	RF3	GET /estudiante/99999 - ID inexistente	Respuesta 404 con mensaje de error descriptivo	Respuesta 404: No se encontró ningún estudiante con ID 99999	OK
PF06	RF4	GET /estadisticas - Estadisticas generales del dataset	JSON con total_estudiantes=20000, medias de horas estudio, sueno, móvil y productividad	Respuesta 200 con total_estudiantes: 20000 y estadísticas correctas	OK
PF07	RF4	GET /estadisticas/genero - Estadisticas agrupadas por genero	JSON con medias de estudio, nota y productividad para Female, Male y Other	Respuesta 200 con los tres géneros y sus métricas diferenciadas	OK
PF08	RF4	GET /estadisticas/productividad	JSON comparativo con datos de grupos ALTA, MEDIA y BAJA	Respuesta 200 con los tres niveles y sus estadísticas	OK

		- Estadísticas por nivel de productividad			
PF09	RF5	POST /predecir - Predicción con datos completos y validos	JSON con predicción (ALTA/MEDIA/BAJA) y probabilidades por clase	Respuesta 200 con prediccion=ALTA, probabilidades y resumen de datos	OK
PF10	RF5	POST /predecir - Datos incompletos (faltan campos)	Respuesta 400 indicando los campos faltantes	Respuesta 400 con campos_faltantes listando los campos no enviados	OK
PF11	RF6	Entrenamiento del modelo - Ejecución de Proyecto_Principal.py genera modelo Decisión Tree	Modelo entrenado y guardado en modelos/modelo_productividad.pkl sin errores	Ficheros .pkl generados correctamente. Accuracy del modelo mostrado en consola	OK
PF12	RF7	Evaluación del modelo - Métricas generadas tras el entrenamiento	Accuracy, classification report y matriz de confusion mostrados en consola	Métricas generadas: accuracy ~81%, F1-score por clase ALTA/MEDIA/BAJA	OK
PF13	RF8	GET /docs - Documentación Swagger de FastAPI	Interfaz Swagger UI cargada correctamente en el navegador con todos los endpoints	Swagger UI accesible en http://localhost:5000/docs con 11 endpoints documentados	OK
PF14	RF9	Todas las respuestas de la API devuelven formato JSON	Cada endpoint devuelve Content-Type: application/json	Verificado en todos los endpoints: respuestas en JSON valido estructurado	OK
Requisitos Adicionales					
PF15	RA1	GET /predicciones - Historial de predicciones (READ)	JSON con total de predicciones, resumen por clase y ultimas 50 predicciones	Respuesta 200 con historial completo y resumen estadístico correcto	OK
PF16	RA1	PUT /estudiante/100 - Actualización parcial de un estudiante (UPDATE)	Respuesta 200 confirmando la actualización de los campos enviados	Respuesta 200 con confirmación. Cambios verificados con GET posterior	OK
PF17	RA1	DELETE /estudiante/200 - Eliminación de estudiante (DELETE)	Respuesta 200 confirmando eliminación. GET posterior devuelve 404	Respuesta 200 con mensaje confirmatorio. GET devuelve 404 correctamente	OK
PF18	RA4	POST /consejo - Consejos con IA (Groq) con datos validos	Respuesta 200 con 3 consejos personalizados en español, modelo y proveedor	Respuesta 200 con consejos generados por LLaMA 3 via Groq en español	OK
PF19	RA4	POST /consejo - Sin GROQ_API_KEY configurada	Respuesta 500 con mensaje indicando que falta la API Key	Respuesta 500: No se encontró la API Key de Groq. Error controlado	OK

3. Pruebas No Funcionales

Las pruebas no funcionales verifican aspectos de calidad del sistema: rendimiento, robustez, usabilidad y reproducibilidad del entorno.

ID	Categoria	Descripcion de la prueba	Resultado esperado	Resultado obtenido	Estado	Obs.
Requisitos No Funcionales						
PNF01	RNF1	Correcta ejecucion - El proyecto arranca y la API responde sin errores en local y en Docker	Servidor operativo sin errores en consola. Todos los endpoints responden	API arrancada correctamente. Sin errores en consola. Endpoints responden	OK	
PNF02	RNF2	Organizacion del codigo - Estructura de carpetas correcta (data/, modelos/, graficos/, documentacion/)	Carpetas separadas para datos, modelos, graficos y documentacion	Estructura verificada: carpetas presentes y cada archivo en su ubicacion correcta	OK	
PNF03	RNF3	Usabilidad - Swagger UI permite probar los endpoints sin herramientas externas	Interfaz /docs cargada y todos los endpoints probables desde el navegador	Swagger UI operativa. Endpoints ejecutables directamente desde http://localhost:5000/docs	OK	
PNF04	RNF4	Reproducibilidad - Proyecto ejecutado en entorno limpio siguiendo el manual de despliegue	Proyecto funciona en otro ordenador siguiendo los pasos del manual	Verificado con Docker: docker build + docker run reproducen el entorno completo	OK	
PNF05	RNF5	Rendimiento - Tiempo de respuesta de GET /estadisticas	Respuesta en menos de 2 segundos	Respuesta obtenida en aprox. 150-300ms en local	OK	
PNF06	RNF5	Rendimiento - Tiempo de respuesta de POST /predecir	Prediccion devuelta en menos de 1 segundo	Respuesta obtenida en aprox. 50-100ms en local	OK	
PNF07	RNF6	Mantenibilidad - El codigo incluye comentarios explicativos en funciones y secciones clave	Comentarios presentes en Proyecto_Principal.py y api.py explicando cada fase y endpoint	Verificado: comentarios de bloque en cada fase CRISP-DM y en cada endpoint de la API	OK	

PNF08	RNF7	Escalabilidad - La estructura permite anadir nuevos endpoints sin modificar los existentes	Nuevo endpoint anadido (POST /consejo) sin alterar el funcionamiento de los demas	Endpoint /consejo anadido correctamente. El resto de endpoints siguen operativos	OK	
PNF09	RA2	Persistencia - Los cambios via PUT y DELETE persisten en SQLite tras reiniciar la API	Datos modificados se mantienen tras reinicio del servidor	Verificado: cambios en estudiantes.db persisten correctamente tras reiniciar la API	OK	
PNF10	RA3	Dockerizacion - docker build construye la imagen sin errores y docker run levanta la API	Imagen construida sin errores. API disponible en localhost:5000	Build completado correctamente. API operativa en contenedor Docker en puerto 5000	OK	

4. Resumen de Resultados

Tipo	Total pruebas	Pruebas OK	Pruebas KO
Funcionales (RF1-RF9)	12	12	0
No funcionales (RNF1-RNF7)	7	7	0
Requisitos Adicionales (RA1-RA4)	**5**	**5**	0

5. Conclusiones

El sistema ha superado la totalidad de las 29 pruebas definidas, cubriendo todos los requisitos funcionales (RF1-RF9), no funcionales (RNF1-RNF7) y adicionales (RA1-RA4). La API REST con FastAPI es robusta, persiste datos en SQLite, se despliega en Docker, dispone de documentación Swagger y consume la API externa de Groq. No existen incidencias pendientes. El sistema está listo para su entrega y despliegue.